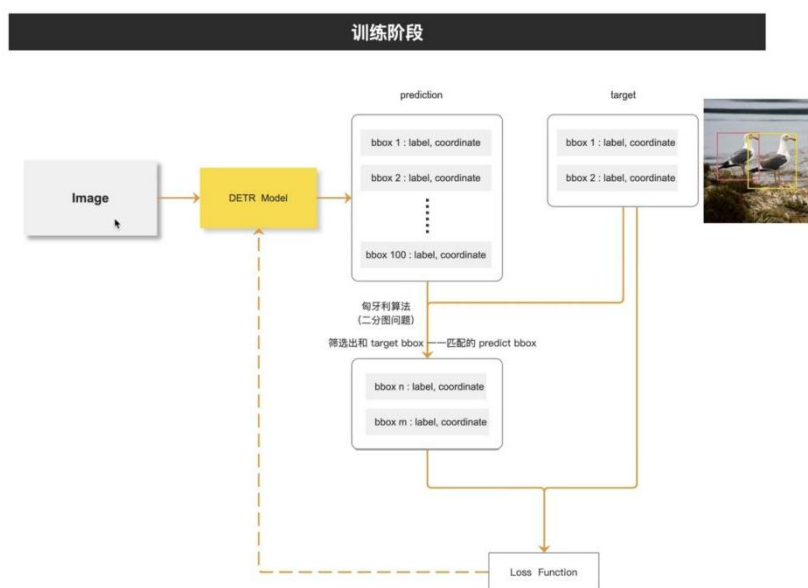


首先，DETR 是一个端到端的网络，不用手动设计锚框。

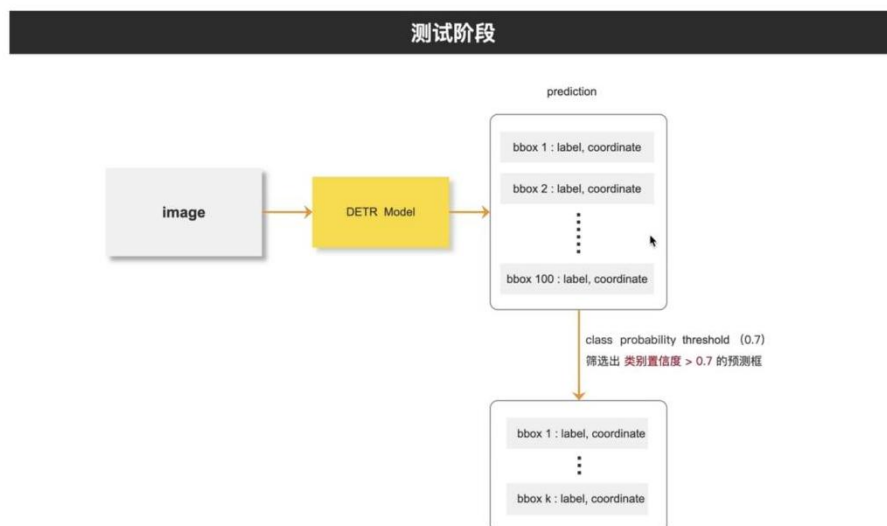
下面是训练阶段的框架：

将一张图像列入 DETR 模型，会得到 100 个预测框，其中包含了这 100 个预测框的类别信息和坐标信息。100 这个数值是一个超参数，因为大部分图像中的 object 的数量都不会超过 100 个。通过标注信息知道这个图像有两个 object，我们使用匈牙利算法从预测出的 100 个候选框中筛选出 2 个预测框。这两个预测框和两个标注框是一一对应的关系。再使用筛选出的这两个预测框和这两个标注框一起去计算损失，反向传播去优化模型参数。



下面是测试阶段的框架：

同样的，通过网络预测出 100 个预测框后，将这 100 个预测框的类别的置信度和类别置信度阈值进行对比，大于阈值的预测框进行保留，小于的就丢弃。



损失函数：是一个基于集合的全局损失，通过二分类匹配得到独一无二的预测结果。这就是上面提到的在训练阶段通过匈牙利算法去解决这个二分图匹配问题，然后我们会得到和标注框一一匹配的独一无二的预测结果。

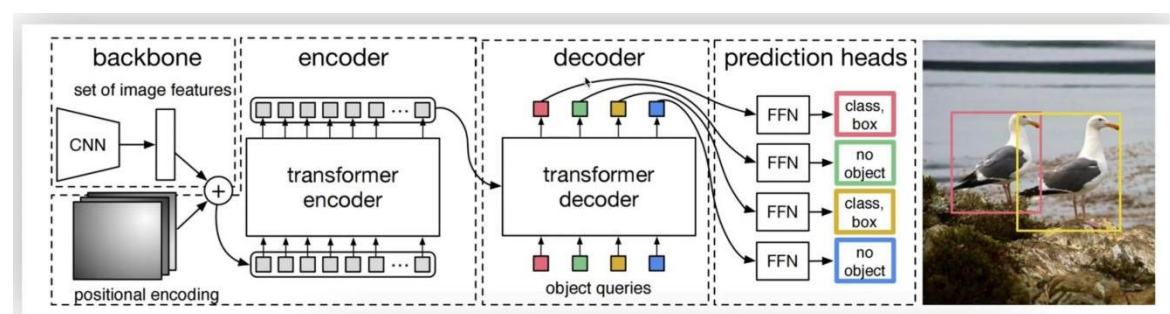
其次，对于 transformer 的 encoder decoder 框架。在 decoder 模块中，设定了一个 learn object queries，通过这个 learn object queries，DETR 就可以对 objects 和图像的全局信息的关系进行推理，并行的直接的输出最终的预测结果。

下面是网络结构图：

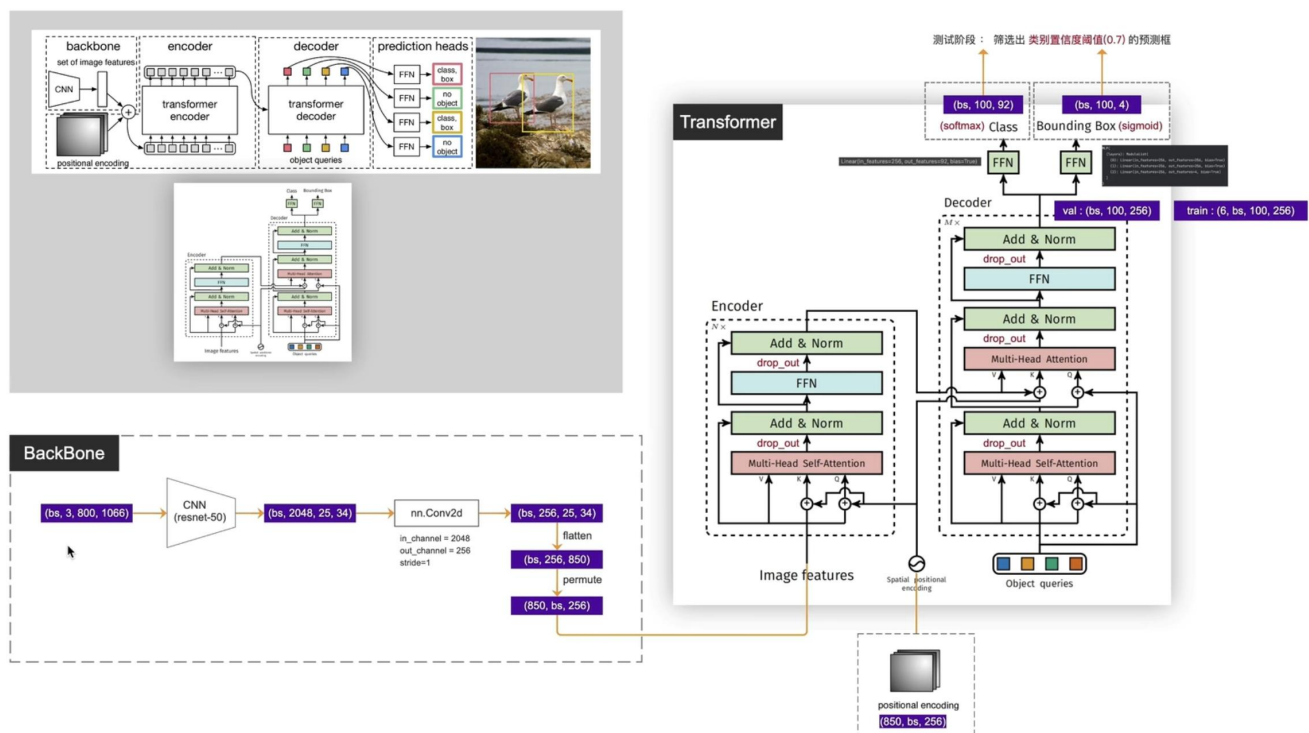
在模型的 decoder 模块里，object queries 是一个可以学习的参数，同时也用其来指定输出（预测框的个数是通过它的尺寸来指定的）。在 transformer 中，输出 token 的个数是等于输入 token 个数的。如果把 object queries 的个数设置为 100，那在输出就可以得到 100 个预测框。

原图像通过 CNN 和 transformer 的 encoder 模块，在这里拿到的就是图像的全局信息，transformer 的 decoder 模块就用于让 object 的预测框和图像的全局信息做自注意力操作，然后得到预测框的集合的输出。还有一点就是 decoder 在生成输出的时候，那些 token 的计算是并行的，不像 NLP。

NLP 是自回归预测，具有顺序性，后面的预测要基于前面的预测结果，所以它没有办法并行。



模型中包括三个部分：卷积网络的 backbone、transformer 结构包括了 encoder 和 decoder 模块以及位置编码、检测头包括两个检测头：一个是用于与预测 bounding box 的类别，另一个是用于预测 bounding box 的坐标。

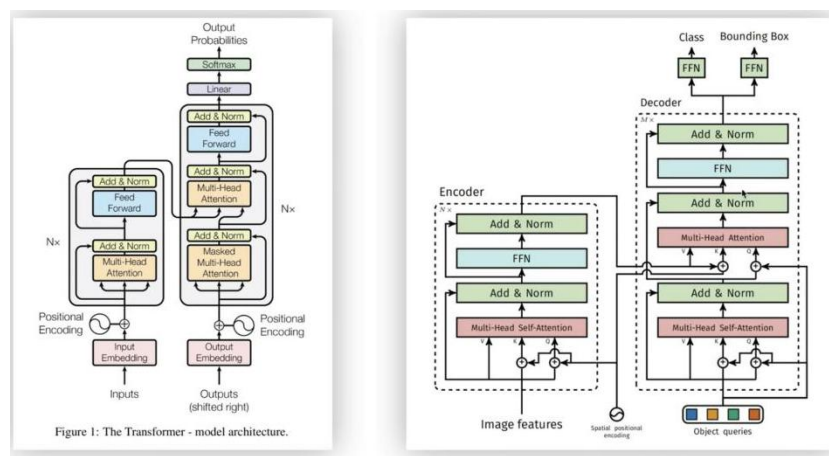


首先拿到一张图像，做完预处理后，图像的尺寸是  $3 \times 800 \times 1066$ ，第一个维度 bs 是 batch size。将图像经过一个卷积网络的 backbone，就可以得到 feature map，其尺寸是  $25 \times 34$ ，下采样了 32 倍，channel 数是 2048。这个 feature map 再经过一个  $1 \times 1$  的卷积层，这个卷积层的目的是为了减少 channel 数。把最后两个维度给 flatten，然后把这两个维度给调换一下。这里的 850 就是要使用 transformer 的 token 的个数，256 就是每一个 token 的特征向量长度。

从 CNN Backbone 得到的 image features 和 position encoding 都要传入到 transformer 的 encoder 模块，这里的位置编码需要在每个 multi head attention 层都要加到 query 和 k 上去。这和标注的呢 transformer 不太一样。

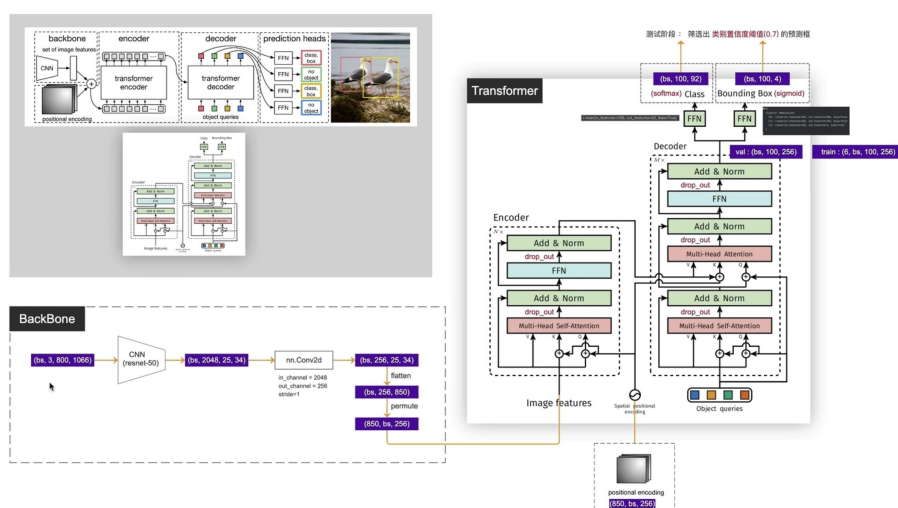
下面是标准的 transformer 和 DETR 的 transformer 机

构。可以看到在标准的 transformer 中，位置编码是直接加载 input 上的，只在这里操作了一次；而在 DETR 的 transformer 中，位置编码是在每个堆叠的 encoder 和 decoder 中都要使用的。比如 N 等于 6，说明 encoder 模块需要堆叠 6 次，对于每个 encoder 中的 multi head attention 层，位置编码都需要直接的加在这个输入上，然后生成 k 和 query。因为输入是一样的，位置编码也是一样的，所以这里得到的 k 和 query 也是一样的。这里的 v 就等于 image features，也就是输入。然后在这 6 个 encoder 中，每个 encoder 都需要这样去操作一次。在 decoder 部分，M 也等于 6，说明 decoder 也要堆叠 6 次，在每个 decoder 模块中，位置编码都需要和 decoder 的输出也就是 memory 相加，然后得到 k。位置编码的操作也是要进行 M 次的。

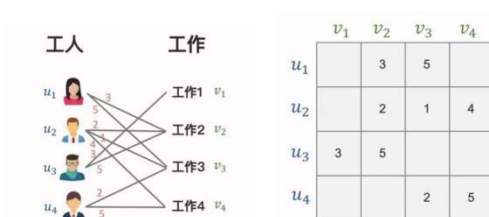


decoder 接受了三个输入，第一个输入是 queries，初始值为 0，shape=(100,256)；第二个输入是 output positional encoding，也就是这个 object queries；第三个输入是 encode memory。encoder 这个部分最后的输出，

包含了图像提取出来的全局信息，经过多个 multi head attention 和 encoder decoder attention，就生成了预测类别标签和 bounding box 的坐标。这是通过这两个检测头得到的。decoder 下边的部分可以把它理解成 multi head self attention 在学习 anchor 的特征；然后上边的部分理解成它在拿到图像的全局特征信息以及 anchor 的特征的基础上预测或者学习图像中的 bounding box 的坐标以及物体的类别。



#### 匈牙利算法：解决二分图匹配问题



某人不能做某项工作（矩阵空白处），  
可以将对应的工资设置为一个非常大的值

```
from scipy.optimize import linear_sum_assignment
```

shape = (100, 2)

```
index = linear_sum_assignment(matrix)
```

输出: index = ([14 33], [1 0])

调用 linear\_sum\_assignment 方法，这个方法能使用匈牙利算法。有 100 行 2 列，然后 2 是 100 和 2 中较小的值，

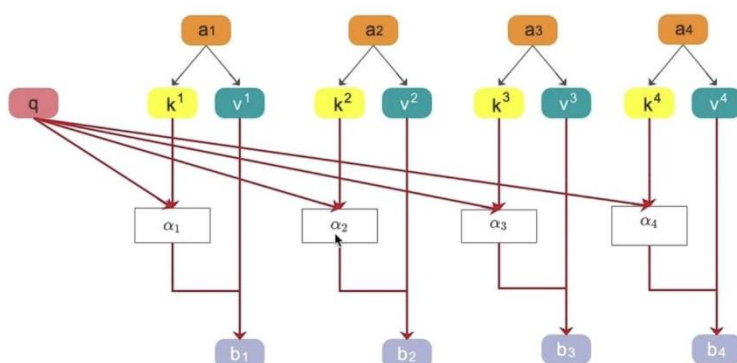


所以拿到的输出结果尺寸就是  $2 \times 2$ 。也就是从这 100 个里面筛选出两个去和这两个进行匹配，得到的结果这四个值都表示索引。输出的 index 表示第一列和第十四行可以进行匹配，然后第零列的和第三十三行的可以进行匹配，这就是一个最优的匹配结果。

# 注意力机制

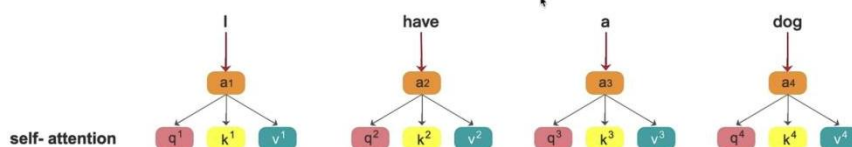


查询 query 和每一个商品的标签 k ( $k_1, k_2, k_3, k_4$ ) 分别去相乘，就能得到  $\alpha_1, \alpha_2, \alpha_3, \alpha_4$ ，这个  $\alpha$  表示它们的相似度，然后再把这个相似度和每一个商品的价值 value 相乘后得到最后这个商品的一个总分  $b_1, b_2, b_3, b_4$ 。



## self-attention 中 self 的理解

self-attention 中的 self，表示 query、key、value 都来自自己，每个 token 都能提取出自己的 query、key、value



Q (query)：模型从 token 中提取出的，对 token 的理解信息。  
用于主动 与其他 token 计算相似程度

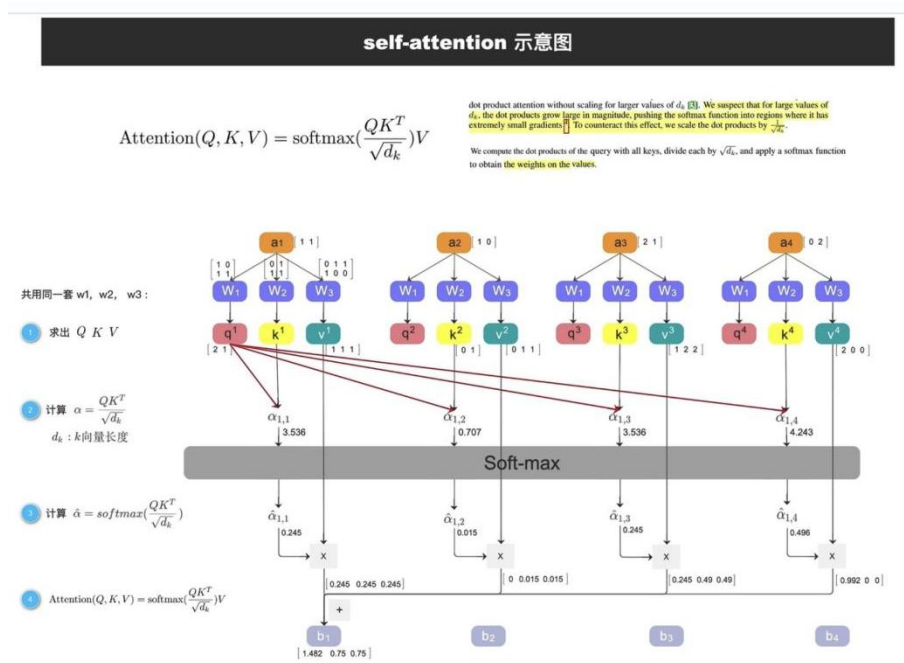
K (key)：模型从 token 中提取出的，与其他 token 的关系信息  
被用于 与其他 token 计算相似程度

V (value)：表示当前 token 的重要程度



转换到自然语言处理领域，通常处理一个给定的句子，不存在外界给定的一个 query 去进行查询，我们想要得到的是这个句子中词与词之间的相关性相似度。我们会让每一个词自己产生一个 query，然后想计算这个词与其他词的一个相似度，就要用这个词的 query 分别和其他词的 k 进行计算，然后得出他们之间的相似程度。

self 就是 query、k 和 value 都是来自于自己的。



公式中的  $d_k$  表示的是 k 的长度，因为 Q 可以和 K 的转置进行相乘，所以 Q 和 K 的维度是完全一样的，V 的维度可以不一样。除以根号是因为：当  $d_k$  的值是一个比较大的值时，即 Q 和 K 的维度是比较长的时候，这个点积就会变得很大，就是 Q 乘 K 的转置这个值会变得很大，这就会将 softmax 给推到具有非常小的梯度的区域中去。经过 softmax 处理后，就得到了 value 的权重，让所有的权重之和等于 1。

$a_3, a_4$  都是 embedding 之后的向量，都是  $1 \times 2$  的向量。

从这几个向量中分别提取出  $q, k, v$ ，提取方式为：让其分别乘以  $w_1, w_2, w_3$ 。这四个  $w_1$  都是同一个  $w_1$ ，因此这里的计算是可以并行的。

self-attention 得到的结果  $b_1, b_2, b_3, b_4$  可以看作是提取出来的 features，这些 features 可以用于下游任务的训练和推理。

得到了  $q, k, v$  后，用  $q_1$  分别取乘以  $k_1$  的转置， $k_2$  的转置， $k_3$  的转置和  $k_4$  的转置。再分别除以根号下  $d_k$ ，这里的  $d_k$  就是  $k$  的维度是等于 2 的。因为  $q$  是一个  $(1, 2)$  的向量， $k$  的转置是一个  $(2, 1)$  的向量，他俩相乘是一个数值，再除以根号下  $d_k$  后还是一个数值。这四个数值称作是相似度分数。再将这四个相似度分数做一个 softmax 的处理，得到四个新的数值分别和  $v_1, v_2, v_3, v_4$  进行相乘再相加，最后得到一个  $1 \times 3$  的向量。因为每一个  $v_1$  都是和一个数值相乘的，所以说  $v_1$  的维度不需要和  $q$  与  $k$  的维度相等。

计算  $b_2$  时，可以把它们视作是提取出来的。

self-attention 代码如下：

```

class Self_Attention(nn.Module):
    def __init__(self, dim, dk, dv):
        super(Self_Attention, self).__init__()
        self.scale = dk ** -0.5
        self.q = nn.Linear(dim, dk)
        self.k = nn.Linear(dim, dk)
        self.v = nn.Linear(dim, dv)

    def forward(self, x):
        q = self.q(x)
        k = self.k(x)
        v = self.v(x)

        attn = (q @ k.transpose(-2, -1)) * self.scale
        attn = attn.softmax(dim=-1)

        x = attn @ v
        return x

att = Self_Attention(dim=2, dk=2, dv=3)
x = torch.rand((1, 4, 2))
output = att(x)

```

代码解释：先创建了一个 Self-Attention 的类，随机一个输入  $x$ ， $x$  的维度是  $(1, 4, 2)$ ，1 是 batch size，4 是指有 4 个 token，2 是每个 token 的强度，然后把这个  $x$  传到对象里边去。 $dk \cdot -0.5$  就是根号下  $dk$  分之一。然后分别使用三个全连接层，从输入里面提取出去  $q, k, v$ ，其中  $q, k$  的输出维度是一定要保证一致的，然后  $v$  的维度不一定要和  $dk$  一样。forward 函数中， $q$  乘以  $k$  的转置，然后做一个 softmax，最后得到的结果和  $v$  进行一个点乘积，得到最终的输出。

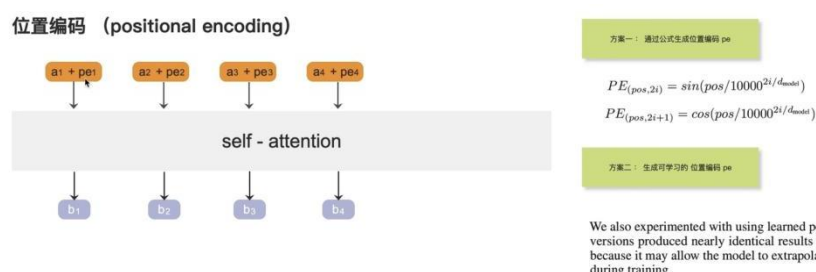
#### 位置编码 positional encoding

self-attention 没有位置信息



位置编码没有位置信息这个弊端，比如输入按照

$a_1, a_2, a_3, a_4$  的顺序，得到的输出是  $b_1, b_2, b_3, b_4$ 。如果把  $a$  的顺序调换一些，对应的输出也会改变。虽然得到的输出  $b$  的顺序也做了相应的变换，但  $b$  内部的数值是没有任何变化的，比如这个  $b_4$  在第一个位置和在第四个位置，他俩内部计算得到的信息是完全一样的。我们希望每个  $b$  提取出来的信息，它内部都是包含这种位置信息的，于是提出了位置编码。

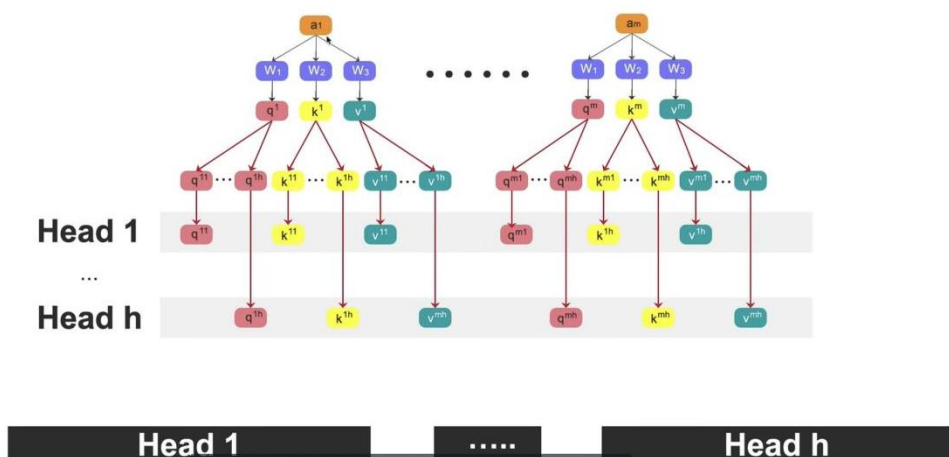


它是加载输入  $a$  上面的，既然它能和  $a$  相加，就说明它的  $a_1$  的尺度是完全一样的，这个位置编码并不是在 self-attention 内部生成的，它是直接加在输入里面的，是通过右边的公式来生成的位置编码。第二个方式是可以生成一个可学习的位置编码，就是说可以随机的初始化一个  $p$ ，然后在 self-attention 内部让他自己去进行学习，学习到它想要的理论位置信息。

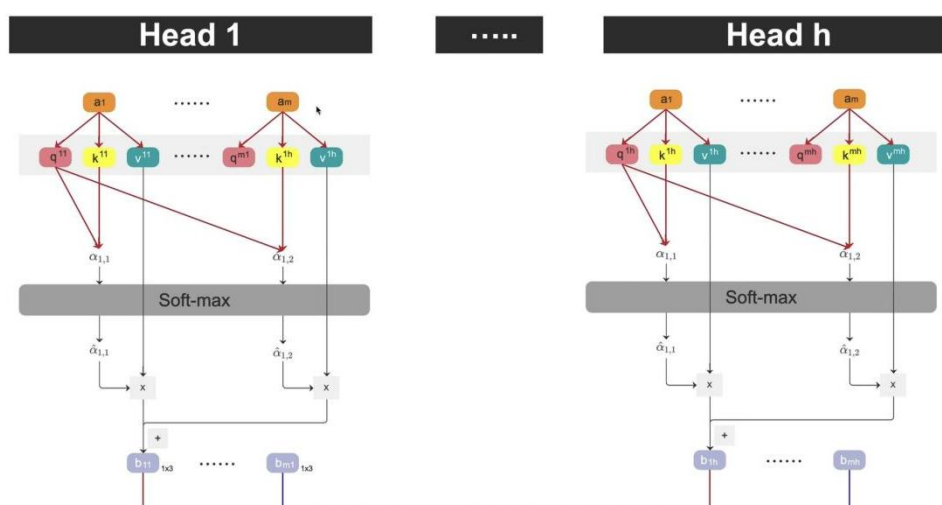
公式中的  $pos$  指的是第几个 token，里面的  $2i$  或者  $2i+1$  指的是每个 token 里面它这个维度的位置。

self-attention 可以看作是 single head attention，在 single head 中  $w_1$  的维度是  $2*6$ ，得出  $q$  的维度就是  $1*6$ 。对应 multi，会把这个  $w$  分成  $h$  份。假设分成了 3 份，对于

head1, w1 的维度就是  $2 \times 2$ ; 对于 head2, w1 的维度就是中间的  $2 \times 2$ ; 对于 head3, w1 的维度就是最后的  $2 \times 2$ 。

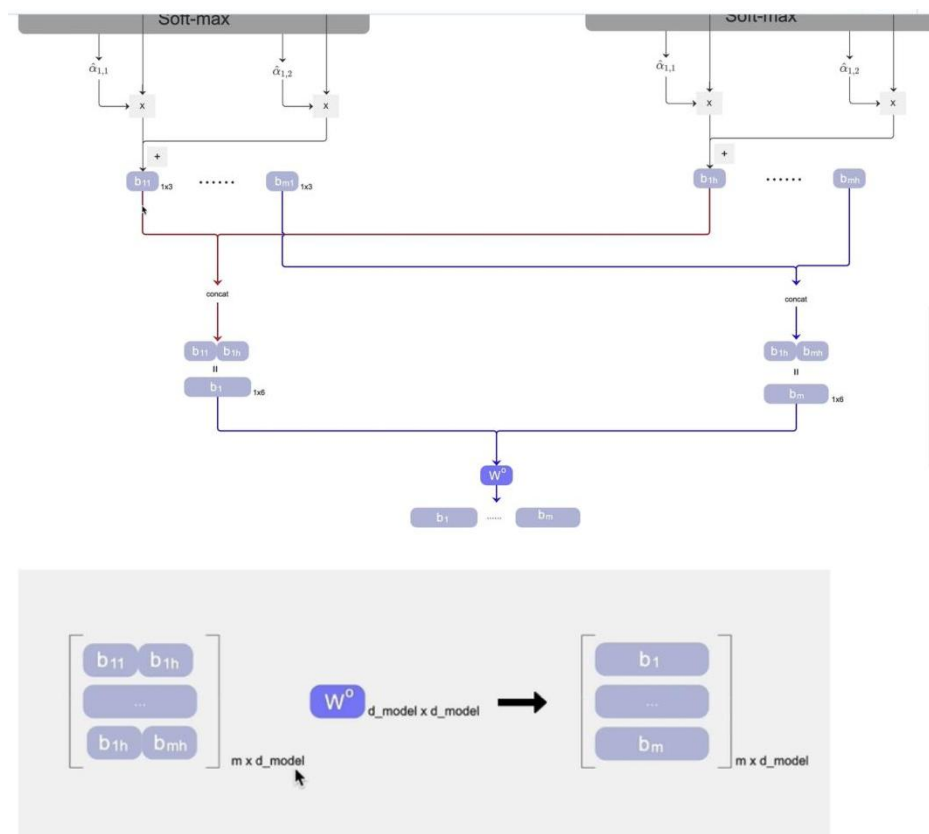


还是将  $a$  如同之前的 single head 一样，从其中提取出  $q, k, v$ ，然后这个  $w$  的维度就是它的  $d_{model}$  的维度，就是全维度。在得到  $q, k, v$  后，把  $q, k, v$  分别分成  $h$  份。第一份就都给到 head1，第二份就都给到 head2..... 最后一份即第  $h$  份给到 head $h$ 。然后分别对于每一个 head 中所有的  $q, k, v$  进行计算。



后面的每一个 head 中的计算过程，就像上面的 self-attention 中的一样。对于每个 head 可以提取出它自

已的  $b_1, b_2, b_3, b_4 \dots b_m$ ，一共有  $m$  个 token。对  $head_h$  也一样，可以提取出  $m$  个  $b$ 。之后将对应位置的  $b$  给 concat 起来。所有 head 中的第一个  $b$  给 concat 起来，组成一个  $b_1$ 。将所有 head 中的一个  $b_m$  给 concat 起来得到  $b_m$ 。再将  $b_1$  到  $b_m$  乘以一个  $w$ ，也就是做一个全连接，得到一个新的  $b_1$  到  $b_m$ 。得到的  $b$  先按列 concat，再按行 concat，就得到了一个维度为  $m$  乘以  $d_{model}$  的矩阵。这个  $m$  就是 token 的个数，然后  $d_{model}$  就是每一个 token 的维度。再乘以一个  $w$  ( $w$  的维度是  $d_{model}$  乘以  $d_{model}$ )，得到的就是最终的结果。这就是 multi-head-attention 的计算过程。





```

class MultiHeadSelfAttention(nn.Module):
    def __init__(self, dim_in, d_model, num_heads=3):
        super(MultiHeadSelfAttention, self).__init__()

        self.dim_in = dim_in # dim_in=2
        self.d_model = d_model # dim_in=6
        self.num_heads = num_heads # dim_in=3

        # 维度必须能被num_head 整除
        assert d_model % num_heads == 0, "d_model must be multiple of num_heads"

        # 定义线性变换矩阵
        self.linear_q = nn.Linear(dim_in, d_model)
        self.linear_k = nn.Linear(dim_in, d_model)
        self.linear_v = nn.Linear(dim_in, d_model)
        self.scale = 1 / sqrt(d_model // num_heads)

        # 最后的线性层
        self.fc = nn.Linear(d_model, d_model)

    def forward(self, x):
        # x: tensor of shape (batch, n, dim_in)
        batch, n, dim_in = x.shape
        assert dim_in == self.dim_in

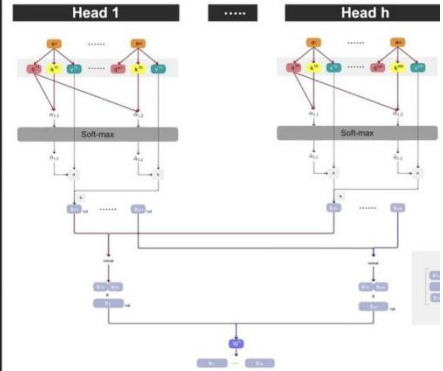
        nh = self.num_heads
        dk = self.d_model // nh # dim_k of each head

        q = self.linear_q(x).reshape(batch, n, nh, dk).transpose(1, 2) # (batch, nh, n, dk)
        k = self.linear_k(x).reshape(batch, n, nh, dk).transpose(1, 2) # (batch, nh, n, dk)
        v = self.linear_v(x).reshape(batch, n, nh, dk).transpose(1, 2) # (batch, nh, n, dk)

        result = (Tensor: [1, 4, 3, 2]) result = (Tensor: [1, 3, 4, 2])
        dist = torch.matmul(q, k.transpose(2, 3)) * self.scale # batch, nh, n, n result = (Tensor: [1, 3, 4, 2])
        dist = torch.softmax(dist, dim=-1) # batch, nh, n, n

```

第一个参数是输入中每个token的维度



```

        # 最后的线性层
        self.fc = nn.Linear(d_model, d_model)

    def forward(self, x):
        # x: tensor of shape (batch, n, dim_in)
        batch, n, dim_in = x.shape
        assert dim_in == self.dim_in

        nh = self.num_heads
        dk = self.d_model // nh # dim_k of each head

        q = self.linear_q(x).reshape(batch, n, nh, dk).transpose(1, 2) # (batch, nh, n, dk)
        k = self.linear_k(x).reshape(batch, n, nh, dk).transpose(1, 2) # (batch, nh, n, dk)
        v = self.linear_v(x).reshape(batch, n, nh, dk).transpose(1, 2) # (batch, nh, n, dk)

        result = (Tensor: [1, 4, 6]) result = (Tensor: [1, 4, 3, 2]) result = (Tensor: [1, 3, 4, 2])
        dist = torch.matmul(q, k.transpose(2, 3)) * self.scale # batch, nh, n, n result = (Tensor: [1, 3, 4, 2])
        dist = torch.softmax(dist, dim=-1) # batch, nh, n, n

        att = torch.matmul(dist, v) # batch, nh, n, dv result = (Tensor: [1, 3, 4, 2])
        att = att.transpose(1, 2).reshape(batch, n, self.d_model) # batch, n, dim_v
        result = (Tensor: [1, 4, 3, 2]) result = (Tensor: [1, 4, 6])
        # 最后通过一个线性层进行变换
        output = self.fc(att) result = (Tensor: [1, 4, 6])

    return output

x = torch.rand((1, 4, 2))
multi_head_att = MultiHeadSelfAttention(x.shape[2], 6, 3) # (6, 3)
output = multi_head_att(x)

```

DETR 的缺点:

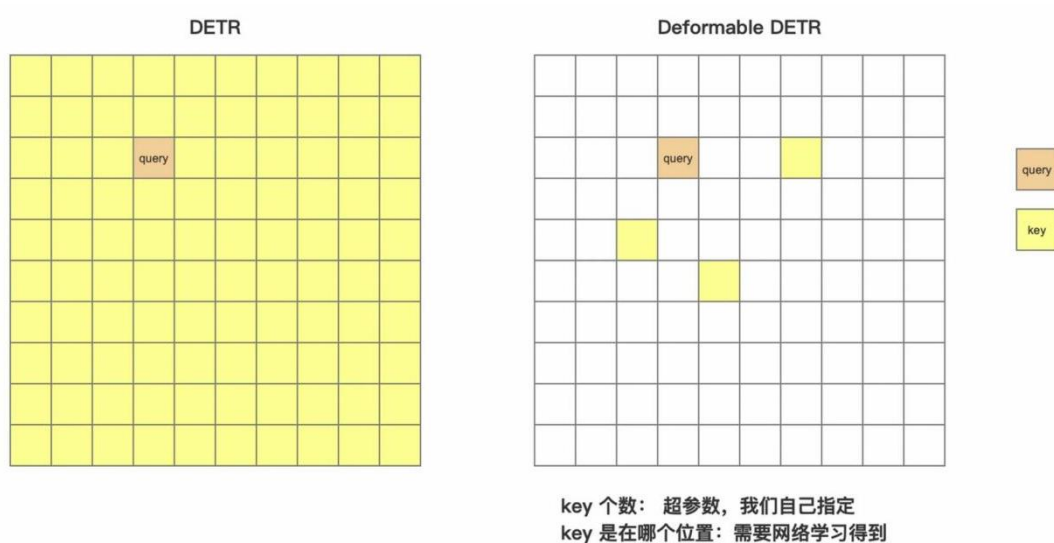
- ①收敛速度慢。
- ②对 feature map 尺寸有限制。

Deformable DETR:

- ①attention modules 只关注参考点周围的一小部分关键采样点。
- ②表现比 DETR 好, 尤其在检测小物体时。
- ③只需要 1/10 的 train epochs。

对于 DETR 来说, 如果图像中的某个像素作为 query, 那么这个图像中的其他所有像素都是 key。query 要和其他的所有 k 来计算相似度。

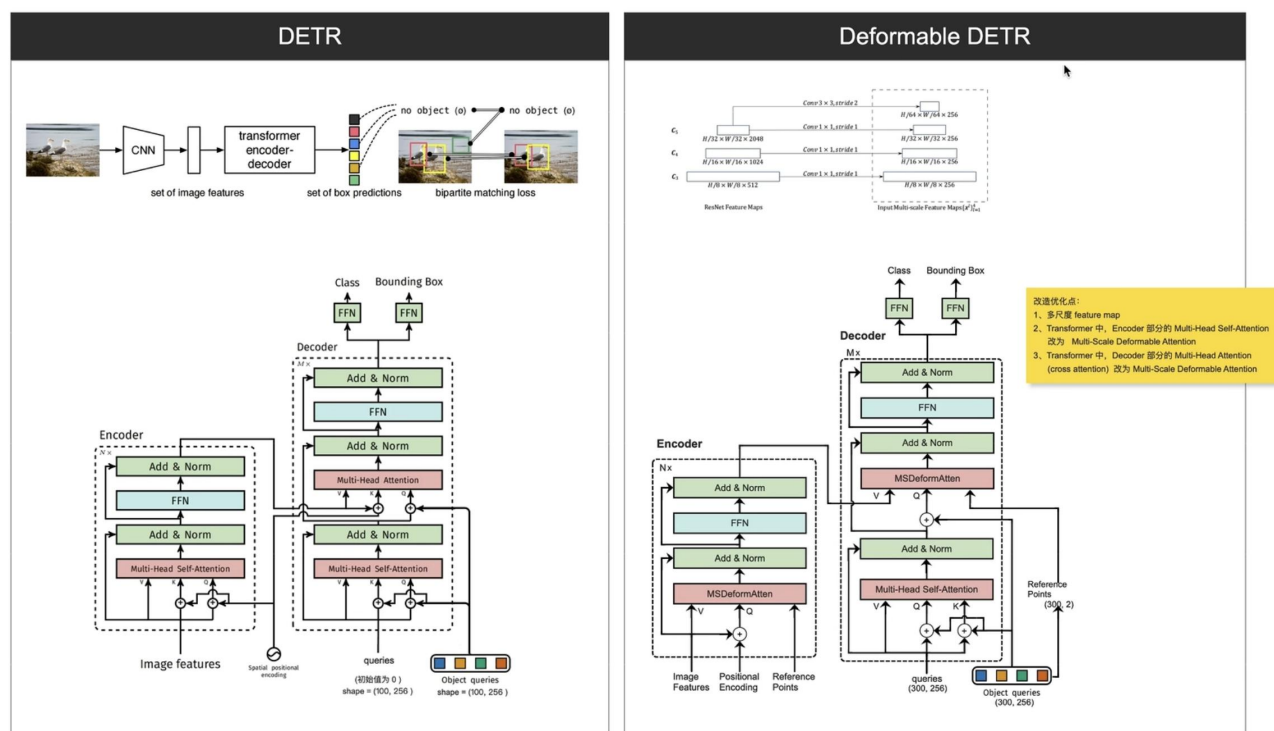
而对于 Deformable DETR: 每个 query 只关注他周围的几个 k。



DETR 的网络结构: DETR 的网络结构先使用 CNN 的 backbone 从原始图像中提取出 feature map, 然后再在这个 feature map 的基础上去做 transformer, 整体大的结构 Deformable DETR 是没有改动的, 都是先去提取出 feature map, 然后再在 feature map 的基础上去做 transformer。Deformable DETR 的第一个改动点是发生在使用 CNN 的 backbone 从原始图像中提取出 feature map 这个部分, DETR 是从原图像中提取出一张 feature map, 也就是 backbone 最后输出的那张 feature map。而 Deformable DETR 则是通过 backbone 去提取出多张 feature map, 就是说 backbone 最后的几个中间层的输出也会提取出来, 如此构成一个多尺度的 feature map。提取多尺度 feature map 的作用主要是去解决检测小物体效果差的问题。因为分辨率大的 feature map 检测小物体比较有优势; 分辨率小的 feature map 检测大物体比较有优势。

self attention 和 cross attention 的区分: 主要是看它输入的 q、k、v 是否来自同一张图像。比如 DETR 中 transformer (Decoder) 的 q、k、v 都来自于同一张 feature map 的, 所以它做的是 self attention。Deformable DETR 中 transformer 输入的 q 是来自下面部分输出的 feature map, 它的 k 和 v 是来自 encoder 的输出的 feature map。所以它的 q 和 k、v 不是来自同一个地方的, 所以这里就是 cross

attention。这部分的改造是为了解决模型收敛慢以及计算量大的问题。



假设现在要计算这个 query 和它周边 3 个 key 的 attention。首先在这个 query 的这个 token 上加上位置编码 (positional encoding)，就得到  $q$ ，然后将这个  $q$  去做一个线性的映射，也就是做一个全连接的操作，就能得到三个位置偏移，也就是公式中的  $\Delta p_{mqk}$ 。然后用已知的 query 的位置加上刚才计算得到的位置偏移量，然后得到三个  $k$  的位置坐标，这对应的就是公式中的  $p_q + \Delta p_{mqk}$ 。然后将  $q$  再做一个线性映射，就得到了 attention weight (就是公式中的  $A_{mqk}$ )。再用  $z_q$  做一个线性映射，就得到了这个 token 的  $v$ ，后面的几个  $v$  是其他的 token。通过计算得到的  $v$ ，用三个  $k$  的位置坐标从 featur map 中取出它们三个对应位置的 value，即

$x_a, x_b$  和  $x_c$ , 得到的这三个值对应的就是这三个  $k$  的  $x(p_q + \Delta p_{mqk})$ 。

